Timing Diagrams

Glossary of Terms

Term	Full Form	What it means
DDR	Double Data Rate	Memory that transfers data on both the rising AND falling edge of the clock — so you get 2x the data per clock cycle compared to SDR (Single Data Rate)
CK_t / CK_c	Clock True / Clock Complement	The two differential clock signals. CK_c is just CK_t inverted. Together they form a differential pair — used because differential signaling is more noise-resistant at high speeds
CMD	Command	Signals sent from the controller to the DRAM telling it what to do (e.g., READ, WRITE, PRECHARGE)
DES	Deselect	A "do nothing" / NOP-like command. Shown in timing diagrams during clock cycles where no meaningful command is being issued. It just means the chip is not selected/addressed
DQ	Data Lines	The actual data bus — this is where your real data bits travel between the controller and the DRAM. "DQ" stands for Data (D) bidirectional (Q is a common notation for a bidirectional signal)
DQS	Data Strobe	A clock-like signal that travels *alongside* DQ. It tells the receiver exactly when to sample the DQ lines. Think of it as a "sample now" pulse. Comes in a differential pair (DQS_t and DQS_c) just like CK
BG / BGa	Bank Group / Bank Group A	DDR4/5 organizes memory into Bank Groups. BGa = Bank Group A. This lets multiple banks operate simultaneously for better throughput
BL	Burst Length	How many data beats are transferred in one READ/WRITE transaction. BL8 = 8 consecutive data words per command
WL	Write Latency	Total number of clock cycles from when the WRITE command is issued to when the first data bit appears on DQ. WL = AL + CWL
AL	Additive Latency	An optional extra delay added on top of CWL. Often set to 0
CWL	CAS Write Latency	The core latency of the write path — cycles from command to first data bit at the DRAM input
CAS	Column Address Strobe	A legacy term; now generally refers to the latency of the read/write data path. CAS Latency (CL) is used for reads

Term	Full Form	What it means
tWPRE	Write Preamble	The time DQS is driven LOW *before* the first DQS toggle/data, to "warn" the DRAM that data is incoming. Acts as a heads-up signal
tWPST	Write Postamble	The brief time DQS is held LOW *after* the last data bit, before being released. Gives the DRAM time to cleanly finish latching the last bit
Din n	Data Input n	The nth data word being written into the DRAM. In a BL8 burst, you'll see Din n through Din n+7 (8 consecutive words)

Key Concepts

Why Two Clocks? (Differential Signaling)

CK*t and CK_c are always opposite each other (one high, one low). The receiver looks at the _difference* between them rather than an absolute voltage. This makes the system immune to noise — if both lines get hit by the same interference, the difference stays clean.

Why DQS exists

At high speeds, the data on DQ lines arrives at slightly different times due to trace length differences, temperature, etc. DQS is sent *with* the data so the receiver knows the exact moment to sample DQ — rather than relying on a separate global clock that might be slightly out of sync.

Center-align vs Edge-align

This is one of the most important concepts in DDR:

- **During WRITES** → DQS is driven by the **controller**. DQS edges are **center-aligned** with DQ (DQS toggles in the middle of each valid DQ window). The DRAM uses those DQS edges to latch data.
- **During READS** → DQS is driven by the **DRAM**. DQS edges come out **edge-aligned** with DQ (both transition at the same time). The controller must internally delay/shift the received DQS to re-center it before sampling DQ. This is what "read training" is about.

Write Burst Operation — Figure 109 Walkthrough

(BL=8, WL=9, AL=0, CWL=9, Preamble=1tCK)

```
T0        → WRITE command issued
            Bank Group Address (BGa) and Bank/Column Address latched

T0→T9     → Write Latency period (WL = AL + CWL = 0 + 9 = 9 cycles)
            Nothing on DQ yet. DRAM is preparing internally.

~T8       → DQS preamble begins (tWPRE)
            DQS goes LOW before toggling, warning the DRAM

T9→T12    → DQS toggles + DQ carries data
            8 data words: Din_n, Din_n+1, ... Din_n+7
            DQS edges are CENTER-aligned with each DQ bit

After T13 → DQS postamble (tWPST)
             DQS briefly stays LOW then releases
```

Summary of Write Flow

1. Controller sends WRITE command with address
2. Waits WL cycles (the latency pipeline)
3. Drives DQS preamble LOW
4. Simultaneously toggles DQS and drives data on DQ
5. After last bit, drives DQS postamble then releases the bus

---

Read Operation — DQS to DQ Relationship (Figure 66)

(BL=8, AL=0, CL=11, Preamble=1tCK)

New Terms for Reads

Term	Full Form	What it means
RL	Read Latency	Total cycles from READ command to first data on DQ. RL = AL + CL + PL
CL	CAS Latency	Core read latency — cycles from command to first data output from DRAM
PL	Parity Latency	Extra cycles added when CA Parity is enabled (error checking on command/address bus). Can be 0
tRPRE	Read Preamble	Same idea as write preamble — DQS goes LOW before toggling to warn the controller that data is coming
tRPST	Read Postamble	DQS held LOW briefly after the last data bit before being released

Term	Full Form	What it means
tDQSQ	DQS to DQ Skew	The maximum time *after* a DQS edge that the DQ data is guaranteed to be valid. In other words: DQS edge fires first, then DQ settles within tDQSQ. This defines how late DQ can be relative to DQS
tQH	DQ Hold Time	The earliest time *after* a DQS edge that DQ is allowed to go invalid/transition. So DQ is guaranteed valid from (DQS edge) until at least tQH after that edge
tDVWd	Data Valid Window (device)	The window of guaranteed valid data per device per UI = tQH − tDQSQ. The bigger this is, the easier it is to sample correctly
tDVWp	Data Valid Window (pin)	Same as tDVWd but measured per individual DQ pin. Can vary pin-to-pin
UI	Unit Interval	One half clock cycle — the time slot for a single data bit on the DDR bus. Since DDR transfers on both clock edges, 1 UI = half of 1 CK period
Dout n	Data Output n	The nth data word coming *out* of the DRAM during a read burst
VDDQ	I/O Supply Voltage	The voltage reference for the DQ/DQS output signals. Timings are referenced to this
DLL	Delay-Locked Loop	A circuit inside the DRAM that synchronizes DQS output timing to the CK input. Must be enabled and locked for read timings to be valid

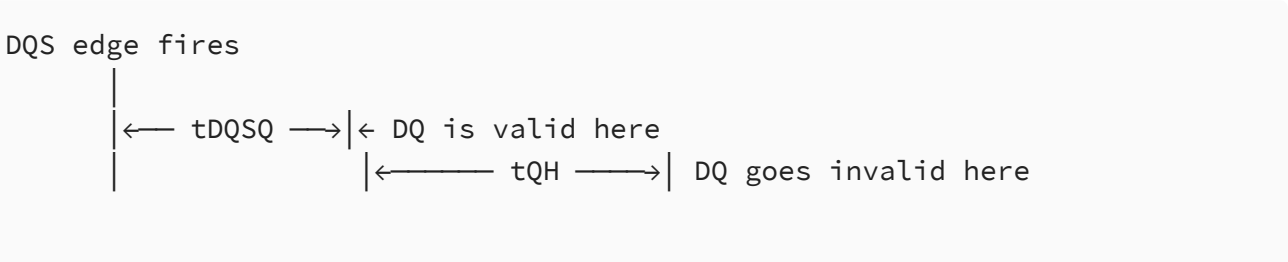
---

Read vs Write — The Critical Difference

In a **write**, the controller drives both DQS and DQ, so it can guarantee they're center-aligned.

In a **read**, the **DRAM drives DQS and DQ**. The DRAM outputs them **edge-aligned** — meaning DQS edges and DQ transitions happen roughly at the *same* time (not centered). The controller cannot simply latch DQ on the DQS edge because DQ hasn't settled yet (it's still transitioning).

This is why tDQSQ and tQH exist — they define the safe window *around* each DQS edge where DQ is guaranteed valid:



```
        │              │                   │
        │         [valid DQ window = tDVW = tQH - tDQSQ]
```

The controller must internally delay its received DQS by roughly half a UI to re-center it within the valid DQ window before sampling. This delay is found during **read training** at boot time.

The Three DQ Rows Explained

The diagram shows DQ three times — this isn't showing different data, it's showing **timing uncertainty**:

- **DQ (Last data)** — worst case where DQ transitions as *late* as possible (bounded by tDQSQ)
- **DQ (First data)** — worst case where DQ transitions as *early* as possible (bounded by tQH)
- **All DQs Collectively** — the overlap of all pins combined, showing the tDVWd window that applies across the whole device

The shrinking of the valid window from tDVWp (per pin) to tDVWd (per device) happens because different pins have slightly different skews — the device-level window is the intersection of all of them.

Read Burst Walkthrough (This Diagram)

```
T0        → READ command issued with Bank/Column Address

T0 → Ta2  → Read Latency pipeline (RL = AL + CL + PL = 0 + 11 + 0 = 11
cycles)
             DRAM is fetching data internally. Nothing on DQS or DQ yet.

~Ta1      → DQS preamble (tRPRE)
             DRAM pulls DQS LOW as a heads-up

Ta2 → Ta5 → DQS toggles + DQ data appears
             8 words output: Dout_n through Dout_n+7
             DQS and DQ are EDGE-ALIGNED (both change together)
             Controller must use a delayed/shifted version of DQS to
sample DQ

Ta5+      → DQS postamble (tRPST)
             DQS briefly held LOW then released
```

Read Burst Operation (Figure — READ Burst, BL8)

(BL=8, AL=0, CL=11, Preamble=1tCK)

New Terms

Term	Full Form	What it means
BC4	Burst Chop 4	A shorter burst mode — only 4 data words instead of 8. The DRAM internally does a BL8 fetch but "chops" it in half, discarding the last 4. Useful when you need less data and want lower latency
BL8	Burst Length 8	Full burst — 8 consecutive data words per READ/WRITE command. Standard mode
A12	Address pin 12	During a READ or WRITE command, this pin is repurposed for burst length control on-the-fly: A12=0 → BC4, A12=1 → BL8. It is NOT used as a column address bit in this context
AUTO PRECHARGE	—	An optional flag (address pin A10) that tells the DRAM to automatically close (precharge) the row after the burst completes, saving you from sending a separate PRECHARGE command

Read Burst Walkthrough

```
T0          → READ command issued
              Bank Group Address (BGa) and Bank/Column Address latched
              A12 sampled → determines BC4 or BL8

T0 → Ta2    → Read Latency pipeline: RL = AL + CL = 0 + 11 = 11 cycles
              DRAM fetches the row data internally. Bus is quiet.

~Ta1        → DQS preamble (tRPRE)
              DRAM pulls DQS LOW — signals to controller "data incoming
soon"

Ta2         → First DQS toggle + first data word (Dout_n) appears on DQ
              DQS and DQ are edge-aligned (transition together)

Ta2 → Ta5   → All 8 data words clocked out: Dout_n through Dout_n+7
              Each DQS edge (rising and falling) transfers one data word
              = 4 DQS cycles × 2 edges = 8 words = BL8 ✓
```

```
Ta6        → tRPST: DQS postamble — held LOW briefly, then
released/tristated
```

How This Differs from the Previous Read Diagram

The previous diagram (Figure 66) was focused on the **DQS-to-DQ timing relationship** — showing tDQSQ, tQH, and the valid data window in detail. That's about signal integrity and how the controller knows *when* DQ is safe to sample.

This diagram is the **system-level view** — showing the full end-to-end burst from command to data, including the CL pipeline, preamble, 8-word burst, and postamble. It's the "big picture" of a read transaction.
